KAROL JASZTAL · Head of Operations & AI Systems Leader

Zero-Trust Autonomous Agent Architecture: Vetting & Governing 11,000+ Multi-Harness Candidate Files

Deterministic Defense-in-Depth, Multi-Dialect PreToolUse Intercepts & Multi-Mode Autonomous Subsystems across 180 Curated Skills & 33 MCP Servers

1. Threat Landscape, Enterprise Vulnerabilities & Defense Blueprint

Empirical threat breakdown: Moving from probabilistic model self-policing to deterministic, out-of-band execution gates

AUDITED PRODUCTION METRICS · 2+ YEARS OPERATIONAL RUNTIME

Enterprise Governance at Scale: The Zero-Trust Defense Funnel

0 Security Incidents · 0 Credential Leaks

11,000+	5,363	50,787	33,108	241	4
Candidate Files Audited	Rejected (~49% Rate)	Managed Vault Files	Lines of System Code	Curated Primitives	Parallel Harnesses

§ 1.1 Executive Threat Analysis: The Enterprise Autonomous Agent Dilemma

Enterprise adoption of autonomous coding agents and workflow orchestrators has reached an inflection point. While autonomous systems yield massive productivity gains, default tool installations expose internal infrastructure to severe security risks. When probabilistic models control shell execution or filesystem access without deterministic gates, traditional application security fails.

1. Arbitrary Shell Execution (Remote Code Execution)

Hallucinated or poisoned model decisions execute destructive shell commands (`rm -rf`, `git push --force`, `DROP TABLE`), wiping uncommitted work or corrupting remote repositories.

2. Credential & Secret Exfiltration

Autonomous agents reading project directories inadvertently ingest sensitive files (`.env`, `.ssh/id_rsa`, cloud provider tokens) and leak them into external LLM provider logs or prompt context.

3. Indirect Prompt Injection & Context Poisoning

Malicious payloads embedded in untrusted web pages, dependency readmes, or Git commit logs hijack model reasoning loops, triggering unauthorized secondary actions under the agent's elevated permissions.

4. Supply-Chain Poisoning & MCP Tool Drift

Third-party Model Context Protocol (MCP) servers and external skills receive silent upstream updates, injecting unauthorized background network daemons, telemetry beacons, or compromised tool definitions.

§ 1.2 The Governance Paradigm: Probabilistic vs. Deterministic Security

Security Dimension	Conventional "Prompt-Guarded" Agents	Karol's Zero-Trust Architecture
Enforcement Layer	System prompt instructions ("*Please do not delete files*")	**Out-of-band PreToolUse hooks compiled in PowerShell / OS**
Failure Mode	Non-deterministic; fails under jailbreaks or long context	**Deterministic; hard-blocked at exit code 2 before shell invocation**
Secret Protection	Agent asked to ignore secrets; files remain readable	**Filesystem regex deny interceptor blocks all `.env*`, `.ssh/` access**
Subagent Boundaries	Monolithic context with all tools available to every agent	**Strict Role-Based Access Control (RBAC); read-only agents have no bash tools**
Supply-Chain Updates	Direct auto-updates from external git repositories	**Radar v2: 3-way merge review, immutable snapshot diffing & human sign-off**

§ 1.3 Four Concentric Rings of Defense: End-to-End Operational Topology

RING 1 Intake

Static Linter Gate

Pre-commit AST inspection, regex scanning for subprocess spawns, YAML frontmatter validation & mojibake encoding audits.

RING 2 Package

Canonical Library

Harness-neutral package layout (`SKILL.md` + `_arsenal.yml`), deploy renderers, and zero vendor lock-in.

RING 3 Runtime

PreToolUse Hooks

Multi-dialect interception: `security-gate` regex denylists, `secrets-guard`, and least-privilege permission filters.

RING 4 Provenance

Radar v2 Engine

Continuous watch-universe monitoring, snapshot materialization in `raw/radar/` & mandatory human 3-way merge reviews.

2. Ingestion, Static Analysis Gate & Multi-Mode Subsystem Anatomy

Automated intake pipeline: Vetting 11,000+ candidate files, enforcing ~49% security rejection, and architecting multi-mode operational engines

§ 2.1 The Ingestion Protocol & Harness-Neutral Packaging Contract

External skills and Model Context Protocol (MCP) servers collected from upstream ecosystems (including Anthropic's Knowledge Work Plugins repository, community repositories, and open MCP directories) never enter production uncurated. Every candidate is normalized into Arsenal's canonical package standard:

Canonical Package Layout

```
library/skills/<item-name>/
├── SKILL.md       # Harness-neutral markdown instructions
├── _arsenal.yml   # Provenance, semver, deploy targets
├── changelog.md   # Reverse-chronological modification log
└── references/    # Offline schemas & specialist checklists
```

Mandatory Frontmatter Schema

```
---
name: database-design
description: "Production Postgres optimization and schema modeling."
harness: universal
allowed-tools:
  - Read
  - Glob
  - Grep
triggers:
  - database design
  - schema review
---
```

§ 2.2 Deterministic Static Linters & Encoding Audits (audit-arsenal.ps1)

Arsenal's automated gate utility (`audit-arsenal.ps1`) executes pre-commit static audits across all library assets, enforcing harness-neutrality and byte-exact encoding health:

```
# Extract from scripts/audit-arsenal.ps1 (Universal Harness-Neutrality Gate & Mojibake Check)
$Script:CcTokens = @('Read','Edit','NotebookEdit','Write','Glob','Grep','Bash','Agent','Task','WebSearch','WebFetch')
$Script:DshTokens = @('read_image','subagent_fork','str_replace_editor','subagent_type','tools.skill')
$Script:PathTokens = @('.claude/','.claude\','~/.claude','$HOME/.claude','.dsh/','.dsh\','$DSH_HOME','.agents/','~/.agents')
$Script:HarnessAllowed = @('universal','claude-code','dsh','antigravity','gemini','openai')

# Gate mode: Exit 1 blocks commit if any non-neutral tool tokens or character corruptions exist
if ($Gate -and ($FmErrors -gt 0 -or $ScriptErrors -gt 0 -or $MojibakeHits -gt 0)) {
    Write-Error "PRE-COMMIT GATE FAILED: Found $FmErrors frontmatter defects and $MojibakeHits encoding hits."
    exit 1
}
```

§ 2.3 The Vetting & Rejection Taxonomy: Empirical Funnel Analysis

5,363 Rejected Files | 11,000+ Files Audited

To ensure zero contamination, rejected candidates are archived in `rejected/` alongside an immutable record of failure reasons. Nearly 50% of external candidate files fail security or architectural criteria:

Category	Identified Threat Vector	Real Upstream Candidate	Vetting Rationale & Operational Defense
`Category A` Orphaned Platform Dependencies	Implicit dependency on external enterprise SaaS or proprietary SDKs.	`account-research (Common Room SDK)`	**REJECTED:** Requires active third-party SaaS credentials; cannot execute without Zoom SDK or Common Room. Replaced by local CRM database query tools.
`Category B` Arbitrary Subprocess Spawning	Scripts attempting un-intercepted OS execution bypassing PreToolUse hooks.	`community-shell-helper`	**REJECTED:** Contained Python subprocess invocations (`subprocess.Popen(shell=True)`) capable of executing unmonitored bash strings.
`Category C` Unsandboxed Network Exfiltration	Hardcoded outbound HTTP requests without proxy or loopback isolation.	`web-enrichment-crawler`	**REJECTED:** Attempted outbound telemetry beacons to unauthenticated endpoints. Exposes sensitive codebase snippets to third-party logs.
`Category D` Monolithic Harness Lock-in	Proprietary harness syntax violating cross-platform portability.	`claude-only-task-runner`	**REJECTED:** Employed non-standard macros incompatible with Google Antigravity and DeepSeek Harness. Re-architected as canonical skill or discarded.
`Category E` Excessive Token Surface (Bloat)	Uncurated documentation dumping 50,000+ tokens into context on every call.	`full-aws-sdk-reference`	**CONVERTED:** Excluded from static prompt context. Re-indexed as an on-demand Model Context Protocol (MCP) semantic lookup tool.

§ 2.4 Production Subsystem Anatomy: Naive Prompts vs. Multi-Mode Engines

Representative Case Study (1 of 180 Skills)

A common industry misconception assumes AI agent "skills" are merely 20-line prompt markdown templates. In Karol's architecture, production skills are **fully orchestrated multi-file operational subsystems** featuring state-machine entry modes, embedded scripts, specialist checklists, and regression gates.

Naive Industry Approach: The Fragile 20-Line Prompt

`Single Flat File · Uncontrolled`

A flat markdown file containing instructions like "*Please review this code for bugs*". Lacks state tracking, diagnostic heuristics, specialized review checklists, and test fixtures. It fails silently under complex multi-file engineering tasks and cannot verify its own findings.

```
# Generic Toy Skill (Single File: prompts/code-reviewer.md)
name: code-reviewer
description: "Reviews code for bugs"
---
You are an expert programmer.
Review the following code and list any bugs:
1. Check syntax errors
2. Check logic errors
3. Give suggestions
```

✗ Zero verification gates ✗ No state tracking ✗ No specialized domain checklists ✗ Single-turn fragility
✗ Non-reproducible outputs

Representative Exemplar: How Production Skills are Architected in Karol's Setup

`Concrete Case Study: 1 of 180 Skills` `46 Files · 6 Operational Modes`

To illustrate the architectural difference between naive prompt engineering and enterprise-grade autonomous systems, below is a concrete dissection of **one single skill** from Karol's 180-skill production library (`review-fix-technical-code`). Rather than a text prompt, each production skill is an entire codified engineering department complete with state-machine routing, domain checklists, automated verification scripts, and replay harnesses:

Part A: 6 State-Machine Operational Modes (Execution Routing) `SKILL.md Orchestrator (634 lines)`

The skill orchestrator routes invocations through deterministic execution modes with strict entry criteria, preventing open-ended or hallucinated agent loops:

`Mode 1: Pre-Merge PR Review`
AST diff analysis, regression baselines, parallel specialist checklists, independent cold-reviewer consensus.

`Mode 2: Fix Reviewer Findings`
Root-cause hypothesis → minimal surgical diff → real terminal evidence output → cold recheck verification.

`Mode 3: Live Crash / Root-Cause Triage`
Exception frame demangling, 5-Whys systematic failure isolation, and reproduction fixture before touching code.

`Mode 4: App-Level QA (Browser / CLI)`
Full-stack exploratory verification, headless Playwright browser automation, console errors and network audits.

`Mode 5: Ticket Pickup & Execution`
Durable tracker integration (Questimus/Jira), acceptance criteria parsing, and red-green-refactor TDD test cycles.

`Mode 6: Pattern Backlog Audit`
Cross-issue retrospective scanning, systemic defect clustering, and architectural pattern library harvesting.

Part B: The Complete 46-File Architecture of this Exemplar Subsystem

All 46 Files Disclosed · Zero Ellipses

Every file in this production subsystem serves an explicit operational purpose across root analysis, language patterns, reproducible test replays, automated lint scripts, and 11 domain specialist review checklists:

```
review-fix-technical-code/ (Unabridged 46-File Production Subsystem Tree)
├── SKILL.md                          # 634-line core engine: 6 operational modes & state machine orchestrator
├── _arsenal.yml                      # Package metadata, semver, provenance & multi-harness target mappings
├── changelog.md                      # Immutable audit ledger of subsystem modifications across releases
├── hitl-loop.template.ps1            # Human-in-the-loop checkpoint runner (PowerShell / Windows environment)
├── hitl-loop.template.sh             # Human-in-the-loop checkpoint runner (POSIX Bash / Unix fallback)
├── analysis/                         # Deterministic Diagnostics & Fault Isolation Engines (2 files):
│   ├── root-cause.md                 # 5-Whys systematic failure hypothesis builder and diagnostic matrix
│   └── stack-trace.md                # Exception frame demangling & source line correlation analyzer
├── patterns/                         # Technology-Specific Audit & Fix Playbooks (9 files):
│   ├── database.md                   # Connection pooling, transaction boundaries & query deadlock recovery
│   ├── docker.md                     # Multi-stage cache invalidation, signal traps (SIGTERM) & hygiene
│   ├── git.md                        # Merge conflicts, rebase branch drift & detached HEAD state recovery
│   ├── go.md                         # Goroutine leaks, channel deadlocks & memory profiling playbooks
│   ├── network.md                    # DNS resolution timeouts, TLS handshake drops & socket exhaustion
│   ├── nodejs.md                     # Event-loop lag, unhandled promise rejections & stream backpressure
│   ├── python.md                     # GIL bottlenecks, async event-loop blocking & type drift mitigations
│   ├── react.md                      # Hook dependency cycles, hydration mismatches & re-render churn
│   └── rust.md                       # Lifetime bounds, borrow checker conflicts & unsafe block audits
├── replay/                           # Regression Prevention & Solution Harvesting (2 files):
│   ├── README.md                     # Replay engine contract & fixture reproduction execution guide
│   └── solution-template.yaml        # Declarative failure replay & regression test specification fixture
├── scripts/                          # Automated Verification & Static Metric Tooling (2 files):
│   ├── lint_runner.py                # Multi-linter execution pipeline with unified exit code aggregation
│   └── type_coverage.py              # Strict TypeScript / MyPy type coverage metric enforcement gate
├── sections/                         # Modular Operational Procedures & Triage Gates (14 files):
│   ├── app-qa.md                     # Full-stack manual & automated verification runbooks
│   ├── checklist.md                  # Universal pre-fix diagnostic and post-fix validation gate
│   ├── ci-failure-playbook.md        # Deterministic CI runner log triage & failure isolation procedure
│   ├── cold-recheck-prompts.md       # Zero-context adversarial prompts for cold audit reviewer agents
│   ├── design-checklist.md           # UI visual regression, WCAG accessibility & token fidelity checks
│   ├── detect-base-branch.md         # Git branch topology detection & merge-base identification
│   ├── diff-analysis.md              # AST-aware diff inspection for unintended side effects
│   ├── greptile-triage.md            # External AI code-review comment parsing & normalization
│   ├── live-bug.md                   # Emergency production defect stabilization & hotfix protocol
│   ├── pre-merge-gate.md             # 4-stage blocking gate before branch merge authorization
│   ├── qa-health-score.md            # Quantitative code health & risk scoring algorithm
│   ├── regression-test-procedure.md  # Regression suite execution & boundary assertion runbook
│   ├── test-bootstrap.md             # Test harness environment initialization & container orchestration
│   └── ticket-format.md              # Standardized durable issue schema for issue trackers
├── sections/specialists/             # Deep Technical Domain Audit Checklists (11 files):
│   ├── api-contract.md               # OpenAPI schema drift, backward compatibility & breaking change guard
│   ├── comments.md                   # Documentation integrity, stale docstrings & comment drift
│   ├── data-migration.md             # Zero-downtime database migrations, rollbacks & lock safety
│   ├── error-handling.md             # Exhaustive error propagation, typed errors & fault isolation
│   ├── maintainability.md            # Cyclomatic complexity, code duplication & modular separation
│   ├── performance.md                # N+1 query detection, memory allocation & hot-path optimization
│   ├── red-team.md                   # Adversarial injection, privilege escalation & attack vectors
│   ├── security.md                   # Secret leaks, OWASP Top 10, CORS & input sanitization
│   ├── test-coverage.md              # Strict branch & condition coverage verification
│   ├── testing.md                    # Flaky test mitigation, mock boundaries & assertion quality
│   └── type-design.md                # Sound typing, discriminated unions & any/unknown defense
└── templates/                        # Audit Documentation & Evidence Output Schemas (1 file):
    └── qa-report-template.md         # Formal cryptographic evidence & sign-off audit report
```

✓ 1 of 180 curated skills in library ✓ Deterministic root-cause triage ✓ 11 domain specialist checklists
✓ Evidence-backed verification loop ✓ Universal across all 4 harnesses

3. Runtime PreToolUse Interception & Deterministic Hook Architecture

Multi-dialect PreToolUse lifecycle hooks enforcing cross-platform denylists (PowerShell + Unix), credential sandboxing & mutation telemetry

§ 3.1 Multi-Dialect PreToolUse Intercept Architecture

Universal Harness Protocol

Because Karol operates autonomous agents across four distinct runtime harnesses (Google Antigravity, Claude Code, DeepSeek Harness, and Gemini CLI), hooks must not depend on vendor-specific intercept formats. His hooks implement a universal adapter pattern emitting decisions to all three major JSON schemas simultaneously:

```
# Multi-Dialect Hook Output Adapter Function (Production Code: library/hooks/secrets-guard/secrets-guard.ps1)
function Write-Decision {
    param([string]$Decision, [string]$Reason)
    $hookEventName = if ($payload.hook_event_name) { $payload.hook_event_name } else { "PreToolUse" }
    @{
        # Dialect 1: Google Antigravity Format
        decision = $Decision
        reason = $Reason
        # Dialect 2: Claude Code Standard Format
        permissionDecision = $Decision
        permissionDecisionReason = $Reason
        # Dialect 3: DeepSeek Harness Codec Format
        hookSpecificOutput = @{
            hookEventName = $hookEventName
            permissionDecision = $Decision
            permissionDecisionReason = $Reason
        }
    } | ConvertTo-Json -Compress -Depth 5
}
```

§ 3.2 The Execution Gate (security-gate.ps1): Dual-OS Regex Denylist

The `security-gate.ps1` hook intercepts proposed commands in the PreToolUse event. To guarantee security in Windows PowerShell and cross-platform container environments, it evaluates commands against a dual-OS audited denylist before shell invocation:

Windows & Unix Destructive Patterns Blocked

- `rm -rf / rm -fr`
- `Remove-Item -Recurse`
- `sudo rm / chmod 777`
- `ri -r -fo / rd /s /q`
- `git push .*(\+|--force)`
- `git push --force-with-lease`
- `git reset --hard`
- `git clean -fd / -f`
- `git branch -D`
- `git checkout . / restore .`
- `DROP TABLE / DATABASE`
- `TRUNCATE TABLE`
- `DELETE WHERE 1=1`
- `Clear-Disk / Format-Vol`
- `mkfs / dd if=`
- `:(){ :|:& };: (Fork bomb)`

Dual-OS Interception Logic

```
$patterns = @(
    'rm -rf', 'Remove-Item\s+(-Recurse|-Force)',
    'ri\s+.*(-r|-fo)', 'rd\s+/s\s+/q',
    'git push .*(\+|--force|-f\b|--force-with-lease)',
    'git reset --hard', 'DROP TABLE', 'TRUNCATE'
)
foreach ($pat in $patterns) {
    if ($cmd -match "(?i)$pat") {
        $msg = "BLOCKED by security-gate: pattern ($pat)"
        [Console]::Error.WriteLine($msg)
        if (-not $isAntigravity) { exit 2 }
        exit 0
    }
}
```

§ 3.3 The Credential Boundary (secrets-guard.ps1): Tiered Secret Isolation

To prevent credential leaks into model contexts or web logs, `secrets-guard.ps1` intercepts file access tools (`Read`, `Edit`, `view_file`) and enforces a strict three-tier authorization policy:

```
# Production Code: library/hooks/secrets-guard/secrets-guard.ps1
$envExclusions = @(".env.example", ".env.sample", ".env.template")

if (-not ($envExclusions -contains $basenameFull)) {
    # Tier 1 Hard Deny: Environment files with active secrets
    if ($basenameFull -imatch "^\.env(\..+)?$") {
        Write-Decision -Decision "deny" -Reason "Blocked by secrets-guard: credential file pattern (.env)."
        return
    }

    # Tier 1 Hard Deny: SSH directory and private keys
    if ($norm -imatch "(^|/)\.ssh/" -or $basenameNoExt -imatch "^(id_rsa|id_ed25519|id_ecdsa)$") {
        Write-Decision -Decision "deny" -Reason "Blocked by secrets-guard: SSH private key store."
        return
    }

    # Tier 1 Hard Deny: Cloud credentials, session tokens, and cryptographic certificates
    if ($norm -imatch "(^|/)\.aws/" -or $basenameFull -ieq ".claude.json" -or $ext -imatch "^\.(pem|key|pfx)$") {
        Write-Decision -Decision "deny" -Reason "Blocked by secrets-guard: token store / certificate."
        return
    }
}

# Tier 2 Confirmation: Heuristic keyword inspection for non-standard files
if ($basenameFull -imatch "credentials" -or $basenameFull -imatch "secrets") {
    Write-Decision -Decision "ask" -Reason "secrets-guard: filename matches heuristic credential pattern. Confirm before proceeding."
    return
}
```

§ 3.4 Least-Privilege Tool Filter (smart-permission-filter.ps1)

Balances security with developer velocity. Safe, idempotent read operations (`Read`, `Glob`, `read_file`, `search_files`) are silently auto-approved, while file mutations are written to an immutable local audit log (`.agents/hook-log.txt`):

```
# Production Code: library/hooks/smart-permission-filter/smart-permission-filter.ps1
$readonlyTools = @('Read', 'Glob', 'Grep', 'LS', 'read_file', 'list_dir', 'search_files', 'view_file', 'search_web', 'TodoRead', 'TaskGet', 'TaskList')
$writeTools = @('Write', 'Edit', 'MultiEdit', 'NotebookEdit', 'write_to_file', 'replace_file_content')

if ($readonlyTools -contains $tool) {
    Write-Allow # Auto-approve safe read operations with zero latency
}

if ($writeTools -contains $tool) {
    # Record mutation telemetry with UTC timestamp, target tool, and caller context
    $timestamp = (Get-Date).ToUniversalTime().ToString('yyyy-MM-ddTHH:mm:ssZ')
    Add-Content -Path '.agents/hook-log.txt' -Value "$timestamp WRITE $tool"
    Write-Allow
}
```

4. Subagent Boundary Isolation & Least-Privilege Execution Matrix

Role-Based Access Control (RBAC): Architectural tool strip-down, ephemeral git worktrees & isolated throwaway databases

§ 4.1 Principle of Least Privilege in Multi-Agent Systems

In standard agent deployments, a single model session holds all available tools (bash execution, file writes, web fetching, and database connections). Under adversarial injection or model drift, compromising any aspect of the session yields full system control.

Karol solves this by decomposing work across specialized subagents with **physically restricted toolsets**. A research agent analyzing untrusted public documentation is physically incapable of running shell commands or modifying files, neutralizing remote code execution (RCE) vectors by design.

§ 4.2 Comprehensive Subagent Tool Boundary Matrix

Core Fleet Sample (17 Active Roles)

Every declared subagent role in Karol's production system is mapped to an explicit tool grant. No subagent possesses tools outside its verified operational envelope:

Subagent Role	Read Tools	Write Tools	Execution	Web Tools	Spawn	Sandbox & Workspace Isolation
`researcher` Codebase discovery & web analysis	✓	✗	✗	✓	✗	**Inherited** Read-only workspace view. Strictly barred from writing or spawning processes.
`librarian` Asset taxonomy & schema verification	✓	✗	✗	✗	✗	**Read-Only** Curated vault view. Enforces canonical naming and frontmatter contracts.
`code-builder` Single task TDD implementation	✓	✓	✓	✗	✗	**Git Worktree** Fresh context per task. Execution strictly gated by `security-gate.ps1`.
`code-reviewer` Independent cold code review	✓	✗	✗	✗	✓	**Read-Only** Zero mutation tools. Spawns read-only specialist subagents (security, perf).
`security-code-reviewer` Vulnerability & secret scanning	✓	✗	✗	✗	✗	**Audit Sink** Audits PR diffs for injection vectors and secret exposures; never writes files.
`design-reviewer` UI/UX token & accessibility audit	✓	✗	✗	✓	✗	**Playwright View** WCAG 2.1 AA / 2.2 audits via headless browser; output restricted to `design/reviews/`.

Full 17-Subagent Governance Fleet: The remaining 11 specialized subagents (including `test-writer`, `docker-hygiene`, `plugin-validator`, `legacy-analyst`, `version-delta-analyst`, and `german-compliance-reviewer`) operate under identical deterministic tool grants. Read-only audit agents are physically barred from mutating state or invoking unmonitored child processes.

§ 4.3 Workspace Boundary Isolation: Worktrees, Ephemeral Schemas & Ports

Ephemeral Git Worktrees

Implementation tasks execute in separate git worktrees (`git worktree add`). Incomplete edits or unexpected file creation cannot contaminate main branch state.

Throwaway Schemas

Database integration tests run against isolated PostgreSQL schemas provisioned dynamically per test run and dropped immediately upon completion, preventing test-state leakage.

Loopback Port Slots

Services bind exclusively to loopback addresses (`127.0.0.1:7030–7039`). Agents are physically blocked from publishing ports to external 0.0.0.0 interfaces.

5. Supply-Chain Provenance & Radar v2 Automated Drift Detection

Eliminating upstream poisoning: Immutable snapshot materialization, 3-way delta briefs & human-directed merge verification

§ 5.1 The Upstream Dependency Attack Surface

External skills and Model Context Protocol (MCP) packages are living software dependencies. Open-source repositories regularly update prompts, tool definitions, or dependencies. Without cryptographic pinning and drift detection, an upstream maintainer can:

1. Silent Prompt Poisoning
Upstream authors modify system prompt instructions to alter agent behavior or inject hidden instructions.

2. Telemetry Infiltration
New updates quietly add network pings or telemetry trackers that leak local file paths or environment variables.

3. Breaking Schema Drift
Uncoordinated tool argument renames cause downstream autonomous agents to fail mid-execution during production runs.

§ 5.2 Radar v2: 5-Stage Human-Directed Drift & Absorption Protocol

`scripts/radar.ps1`

Karol engineered Radar v2 to monitor the entire monitored repository watch-universe. External updates are never automatically pulled into production; they pass through an immutable snapshot staging pipeline:

1. **SCAN** — Fetches upstream commit tree hash & diff
2. **SNAPSHOT** — Materializes exact snapshot in `raw/radar/`
3. **BRIEF** — Generates 3-way AST & token delta brief
4. **DECIDE** — Human signs off: `clean|absorb|skip`
5. **ABSORB** — Promotes to library & bumps semver

§ 5.3 Radar v2 CLI Command Contract & Audit Trail

The CLI ensures every update carries full traceability back to upstream commit hashes and records human review justification in `_arsenal.yml`:

```
# 1. Scan monitored upstream repository watch-universe for drift
.\scripts\radar.ps1 scan -Repo "https://github.com/DietrichGebert/ponytail"

# 2. View generated delta brief and AST changes for a modified skill
.\scripts\radar.ps1 brief -Item "javascript-testing-patterns"

# 3. Record human verdict following security review
.\scripts\radar.ps1 decide -Item "javascript-testing-patterns" -Verdict absorb -Reason "Security audited: upstream added Vitest 4.0 type definitions"

# 4. Verify staged content against static linters and test suites
.\scripts\radar.ps1 verify -Item "javascript-testing-patterns"

# 5. Finalize promotion, update changelog.md, and bump semver in _arsenal.yml
.\scripts\radar.ps1 absorb -Item "javascript-testing-patterns"
```

6. Empirical Vetting Registry, Primary Case Studies & ROI Scorecard

Audited inventory of 241 active primitives, real-world CVE/exfiltration case studies & executive risk scorecard

§ 6.1 The Complete Arsenal Control Plane: 241 Primitives & 33,100+ Lines of Code

241 Curated Primitives · 50,787 Vault Files

Every production item is maintained as an audited canonical package in `library/`, verified across 4 major commercial harnesses, and tracked with strict semver:

PRODUCTION SKILLS	CURATED MCP SERVERS	AUTONOMOUS SUBAGENTS	LIFECYCLE HOOKS
180 Skills	**33 Servers**	**17 Subagents**	**11 Hooks**
33,108 lines of orchestration code	Filesystem, Postgres, Git, Puppeteer	Least-privilege RBAC tool scoping	PreToolUse, security-gate, secrets

ENGINEERING	GOVERNANCE	SECURITY	DATABASES	UI SYSTEMS	PERFORMANCE
54 Skills	**38 Skills**	**24 Skills**	**18 Skills**	**28 Skills**	**18 Skills**
TS, Python, TDD	Multi-agent loops	GDPR, GoBD, hash	Postgres, SQLite	Tailwind, WCAG 2.2	Ads API, GA4

§ 6.2 Primary Vetting Case Studies: Real Operational Defect Interceptions

Case 1: Anthropic Common Room Plugin (Exfiltration Vector) — REJECTED

Source: `partner-built/common-room/skills/account-research`

Audit Finding: The skill declared an innocent research trigger but contained hard dependencies on external SaaS APIs and unauthenticated network transmission. It routed user company queries through external SaaS proxies without encryption guarantees.

Action Taken: Rejected outright and permanently logged in `rejected/2026-06-30-account-research-skill/CANDIDATE.md`. Replaced with local CRM SQL tooling executing entirely within private loopback containers.

Case 2: Community Web Scraper MCP Server (Local File Protocol Exploit) — REMEDIATED

Source: Open-source community Puppeteer-based Model Context Protocol (MCP) server.

Audit Finding: Server configuration passed default flags `--no-sandbox --allow-file-access-from-files` to headless Chromium. An indirect prompt injection in scraped web content could instruct the browser to navigate to `file:///C:/Users/...` and exfiltrate local files via image pingbacks.

Action Taken: Stripped hazardous flags, sandboxed browser execution to loopback-only requests, and deployed dedicated `ui-ux-reviewer-agent-browser` subagent with strict URL scheme linters.

§ 6.3 Executive ROI & Risk Reduction Scorecard

Audited Operational Baseline (2+ Years)

Risk / Governance Dimension	Unconstrained Deployment	Karol's Zero-Trust Architecture	Audited Operational Outcome
Rogue Destructive Commands	High (LLMs hallucinate destructive shell commands)	Deterministic Multi-Layer Containment (PreToolUse denylist + Worktrees)	**0 Destructive Incidents (100% Intercepted)**
Credential & Token Leaks	High (Agents freely read .env and .ssh folders)	Air-Gapped Two-Tier Secrets Guard (Filesystem block at tool boundary)	**0 Credentials Leaked across all sessions**
Supply-Chain Silent Poisoning	High (Unmonitored external package drift)	Radar v2 Upstream Drift Engine (3-way merge briefs & human sign-off)	**100% Traceability across 11,000+ files**
Cross-Harness Interoperability	Zero (Vendor lock-in to single commercial harness)	Canonical Packaging & Multi-Dialect Hooks (Universal portability)	**100% Parity (Claude Code, DeepSeek Harness, Google Antigravity, Gemini CLI)**